```
    print $refMethod->getReturnType()->getName() . PHP_EOL;
}

Output:
... and methods
  final public prayerOfHealing(): void
```

In addition to the plethora of OOP tools, it is also possible to reflect functions…

```
class Treasure {}
class Party {}
class Dungeon {}

function delve(Party $party, Dungeon $dungeon): Treasure
{
    // adventures go here
    return new Treasure();
}

$refFunc = new ReflectionFunction('delve');

print "Function {$refFunc->getName()}() ".
    "should be called with parameters..." . PHP_EOL;

foreach ($refFunc->getParameters() as $param) {
    print $param->getType()->getName() . ': ';
    print '$'. $param->getName() . PHP_EOL;
}

Output:
Function delve() should be called with parameters...
Party: $party
Dungeon: $dungeon
```

… and even use `ReflectionFunction::invoke()` to call them:

```
var_dump($refFunc->invoke(new Party, new Dungeon));


Output:
object(Treasure)#5 (0) {
}
```